Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

ADAPTIVE REVERSE PROXY WITH LATENCY-AWARE ROUTING

24BAI0117	**SESHRUT JAIN**
24BCE0897	**JOKA SRINIVASA SAI KRISHNA SRIDDA**
24BCE0924	**VAMSI INDRAKANTI**

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering

October 2026

ABSTRACT

Static load-balancing policies such as round robin divide requests equally across back-end servers, so a slow server receives as much traffic as a fast one. This project implements an adaptive reverse proxy that routes each request using measured response time and current load. The proxy times every forwarded request, smooths the samples per server with an exponentially weighted moving average (EWMA, $\alpha = 0.3$), and selects the server with the lowest score, defined as EWMA × (requests in flight + 1). About 10% of requests go to a random healthy server so that a recovered server is tried again, and a server with no latency data is tried first. If a server returns an error or times out (2 s), the proxy retries on the other healthy servers; three consecutive failures remove a server from the pool until a probe succeeds.

The system is built with Next.js 14, React 18 and TypeScript, with three simulated back-ends and a live dashboard. Against round robin, in three runs of 30 s at 10 packets/s with server delays of 80, 150 and 250 ms, adaptive routing sent 59–68% of requests to the fastest server (round robin: 33%) and had 22–29% lower mean proxy-measured latency in every run pair (mean 163 ms versus 223 ms). When one server was slowed to 400 ms, its share of the following 250 requests fell from 64–66% to 13–14%. Taking a server down caused no client-visible failures in two runs. The results come from three runs on one machine with simulated back-ends and are a limited demonstration, not a production benchmark.

CERTIFICATE

This is to certify that the project report titled "Adaptive Reverse Proxy with Latency-Aware Routing" is a bonafide record of the work done by Seshrut Jain (24BAI0117), Joka Srinivasa Sai Krishna Sridda (24BCE0897), Vamsi Indrakanti (24BCE0924) in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

_____ Seshrut Jain (24BAI0117)

_____ Joka Srinivasa Sai Krishna Sridda (24BCE0897)

_____ Vamsi Indrakanti (24BCE0924)

Name and Signature of the Students

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	2
1.	**INTRODUCTION**	6
	1.1 Background and motivation	6
	1.2 Problem statement	6
	1.3 Objectives	6
	1.4 Scope and limitations	6
	1.5 Mapping to course modules and the lab experiment	7
2.	**LITERATURE SURVEY**	8
	2.1 Overview of the existing protocol or scheme	8
	2.2 Review of related work (table: author and year, technique, metrics, merits, gaps)	8
	2.3 Research gap and justification of novelty	9
3.	**PROPOSED SYSTEM**	10
	3.1 System architecture or topology (diagram)	10
	3.2 Baseline mechanism description	10
	3.3 Proposed mechanism: algorithm, pseudocode, flowchart	10
	3.4 Novelty statement (what exactly differs from the baseline)	12
	3.5 Assumptions and design decisions	13
4.	**IMPLEMENTATION**	14
	4.1 Tools, platform, versions and hardware	14
	4.2 Simulation or testbed setup (topology, configuration, parameter table)	14
	4.3 Code structure and explanation of key modules	14
	4.4 Challenges faced and how they were solved	15

Sl.No	Contents	Page No.
5.	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	16
	5.1 Performance metrics and why they were chosen	16
	5.2 Scenarios and parameter values	16
	5.3 Results: tables and graphs, baseline vs proposed (mean of repeated runs)	16
	5.4 Analysis and discussion (reasons, trends, trade-offs)	19
	5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails	20
6.	**CONCLUSION AND FUTURE WORK**	21
	6.1 Summary of work and key findings (quantified)	21
	6.2 Limitations	21
	6.3 Future enhancements	21
7.	**REFERENCES**	22
	References (IEEE style)	22
	Appendix A: Important source code or configuration listings	23
	Appendix B: Additional screenshots, packet captures, raw result tables	24

1. INTRODUCTION

1.1 Background and motivation

A reverse proxy is a server that sits in front of one or more back-end servers and presents a single public address to clients. Clients talk only to the proxy; the servers stay on a private network behind it. This makes the proxy the natural place to balance load, hide server details, cache content and handle failures. A forward proxy is the opposite arrangement: it sits in front of clients and is used for filtering and access control. This project builds a reverse proxy.

Table 1.1: Forward proxy versus reverse proxy

	Forward proxy	**Reverse proxy**
Sits in front of	Clients	Servers
Hides	The client	The servers
Set up by	Client side or organisation	Server owner
Typical use	Filtering, access control	Load balancing, caching

Three standard load-balancing methods are in common use. Round robin gives each server a turn in a fixed order. Least connections sends the request to the server with the fewest open requests. IP hash sends the same client address to the same server every time. None of these chooses a back-end using measured response latency. Round robin is the simplest and the most widely used, but it treats all servers as equal: with two equally selected servers, about half of the requests can end up at the slow one even while the fast one has spare capacity. This is the situation that motivates the project.

1.2 Problem statement

Static load-balancing policies ignore how quickly each server is actually responding. When back-end speeds differ, or change over time, a static policy keeps sending the same share of traffic to the slow server and the average response time suffers. The problem addressed here is to design and implement a reverse proxy that chooses a back-end for every request using recent measured latency and current load, keeps working when a server fails, and makes its behaviour visible in real time.

1.3 Objectives

- Measure the response time of every forwarded request at the proxy.
- Choose the back-end using recent latency and current load, so that faster, less busy servers receive more traffic.
- Keep working when a server fails: retry on other servers, remove a failing server from the pool and bring it back when it recovers.
- Show routing decisions, latency and server health live on a dashboard.
- Compare the adaptive strategy with round robin as the baseline, using the same setup, and report the results with their limits.

1.4 Scope and limitations

The scope is a single proxy instance that routes HTTP requests to three simulated back-end servers. The back-ends are routes inside the same Next.js application, with configurable delays, so that the experiments are repeatable. Routing state (latency averages, health, statistics) is held in memory and is lost on restart. The in-flight count is kept by the simulated back-end, and load is counted both in the measured latency and in the score. Health probes run only when traffic arrives. The experiments are three runs on one machine with simulated servers; they are a limited demonstration and not a throughput benchmark. Section 6.2 lists the limitations in full.

1.5 Mapping to course modules and the lab experiment

The project connects to the Computer Networks Lab through the following topics.

Table 1.2: Mapping of the project to course topics

Course topic	Where it appears in the project
Application layer: HTTP, client–server model, proxies	The proxy receives HTTP requests from clients and forwards them to back-end servers; forward versus reverse proxy (Section 1.1).
Transport layer: round-trip time estimation and timeouts	Per-request round-trip timing, EWMA smoothing of samples (the same idea as TCP's smoothed RTT estimator) and the 2 s request timeout.
Network performance: latency, load, load balancing	Round robin baseline versus latency- and load-aware routing; mean and p95 latency; share of traffic per server.
Reliability and monitoring	Retries, failure counting, health probes, and the live dashboard.

Lab experiment mapped: [Experiment number and title to be filled in by the team].

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

Static policies. Open-source web servers and proxies provide round robin, least connections and IP hash as standard policies. In NGINX, for example, `least_conn` and `ip_hash` are available in the open-source upstream module, while a latency-based `least_time` method is a commercial (NGINX Plus) feature [7]. Round robin is also the baseline in this project.

Smoothing round-trip times. Estimating a delay with an exponentially weighted moving average is a long-established technique in networking. TCP keeps a smoothed round-trip time and a variance estimate to set its retransmission timeout [1], [2]. The estimator needs one stored value per path and reacts to change without keeping a history; the same properties make it suitable for keeping one latency estimate per back-end server.

Load-aware randomised choice. Mitzenmacher showed that sampling two servers at random and sending the request to the less loaded one gives much better balance than a single random choice [3]. Surveys of web-server load balancing classify the approaches by where the decision is made (client, DNS, dispatcher or server) and by the information they use [4].

Latency-aware balancing in production systems. Finagle, Linkerd and Envoy implement a "peak EWMA" balancer that combines a latency estimate with the number of requests in flight. A Linkerd article compared round robin, least loaded and peak EWMA in a Finagle client with slow replicas and reported that peak EWMA performed best [5]. Envoy documents its cost function as the peak-EWMA round-trip time multiplied by (active requests + 1) and warns that hosts that fail fast can look fast, so health checking and outlier detection should remove failing hosts first [6]. The score used in this project has the same shape, EWMA × (requests in flight + 1).

2.2 Review of related work (table: author and year, technique, metrics, merits, gaps)

Table 2.1: Review of related work

No.	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	V. Jacobson, 1988 [1]	EWMA-smoothed round-trip time (with variance) to set the TCP retransmission timeout	RTT, retransmission timeout; BSD TCP implementation	One value per path; adapts to change; no history kept	Protocol timer for one connection, not server selection
2	V. Paxson et al., RFC 6298, 2011 [2]	Standardised SRTT/RTTVAR update and RTO computation	SRTT, RTTVAR, RTO	Simple, widely deployed; fixed smoothing constants	Same: timer for one connection
3	M. Mitzenmacher, 2001 [3]	Power of two choices: sample two servers, pick the less loaded	Queue length, maximum load; analysis and simulation	Large balance gain for little state	Uses load only; no measured latency

No.	Authors, year	Technique	Metrics and tool	Merits	Gaps
4	V. Cardellini, M. Colajanni, P. S. Yu, 1999 [4]	Survey and taxonomy of dynamic load balancing for web-server systems	Server load, response time (survey)	Clear classification of where and how decisions are made	Survey only; no latency-aware proxy implementation
5	Linkerd / Finagle, 2016 [5]	Peak EWMA versus round robin and least loaded in a Finagle client	Latency under slow replicas; Finagle	Production-tested; combines latency and queue depth	Library or service-mesh component; not a small proxy with a visible dashboard
6	Envoy Proxy documentation [6]	Peak EWMA load balancer, cost = RTT × (active requests + 1)	Per-host RTT samples; Envoy HTTP filter and balancer	Latency- and load-aware; documented behaviour	Needs separate HTTP filter and configuration; relies on health checking for failing hosts
7	NGINX documentation [7]	Built-in upstream methods: round robin, least_conn, ip_hash; least_time in NGINX Plus	Connections, response time (least_time)	Mature and fast	Open-source methods do not use measured latency

2.3 Research gap and justification of novelty

Latency-aware balancing is already available in large systems such as Finagle, Linkerd and Envoy, but there it is part of a larger framework and its behaviour is not easy to see or experiment with. The common open-source policies used in a lab (round robin, least connections, IP hash) do not use measured latency. This project does not claim a new algorithm. Its contribution is a small, readable implementation of the latency-and-load idea inside a reverse proxy, together with controlled experiments against round robin and a dashboard that lets a viewer slow down, take down or add a server and watch the routing react. Section 3.4 states precisely what differs from the baseline.

3. PROPOSED SYSTEM

3.1 System architecture or topology (diagram)

The system is a single Next.js application. A React dashboard in the browser sends requests and polls statistics; the proxy API route chooses a back-end with the configured strategy, forwards the request, times it and updates the shared state; the three back-ends are simulated routes inside the same application.

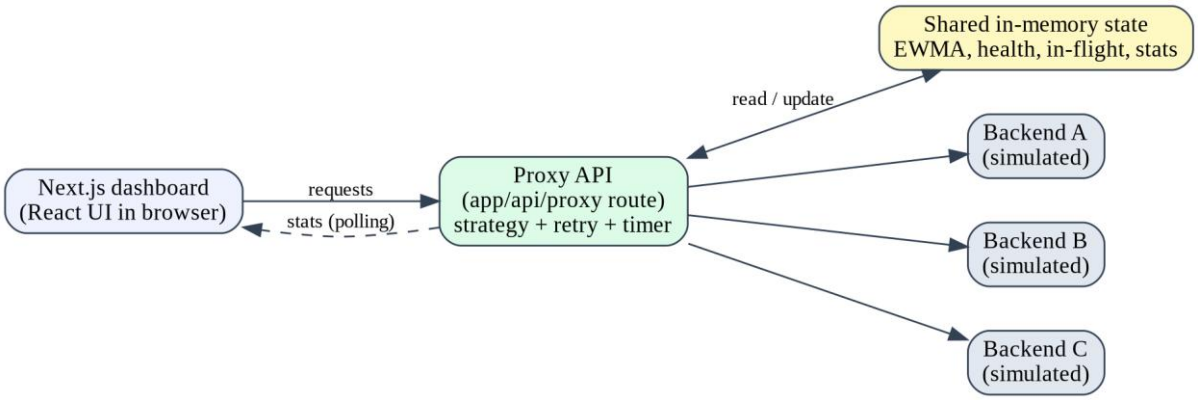


Figure 3.1: System architecture

The request flow has five steps that repeat for every request: (1) the client sends a request to the proxy; (2) the strategy picks a healthy server; (3) the proxy forwards the request and starts a timer; (4) the response returns and the round-trip time is recorded; (5) the EWMA is updated and the response is sent to the client.

3.2 Baseline mechanism description

The baseline is round robin (`lib/strategies/round-robin.ts`). Each healthy server takes a turn in a fixed order, so with three servers each receives one third of the requests, regardless of how fast it responds. It needs no measurement and no state beyond the position in the order, which is why it is simple and predictable, and also why it cannot react when one server is slower than the others.

3.3 Proposed mechanism: algorithm, pseudocode, flowchart

The adaptive strategy (`lib/strategies/lowest-latency.ts`) has three parts.

Latency measurement. The proxy starts a timer before forwarding a request and computes the round-trip time when the response returns. This is the proxy-to-back-end interval (the timer wraps the call to the back-end); it is not the total latency seen by the client.

EWMA smoothing. For each server the proxy keeps one smoothed latency, updated after every response:

$$EWMA_new = \alpha \times sample + (1 - \alpha) \times EWMA_old, \quad \alpha = 0.3$$

The first reply sets the starting value. With α = 0.3, one slow reply moves the average only 30% of the way to the new sample, so single spikes are damped rather than removed, while a real slowdown shows within a few requests.

Selection. For each new request the proxy first checks whether a healthy server has no latency data yet; if so, that server (the least busy one if there are several) gets the request. Otherwise, with probability about 0.9 the proxy chooses the server with the lowest score, and with probability about 0.1 it chooses uniformly at random among the healthy servers:

$$score = EWMA\ (ms) \times (requests\ in\ flight + 1)$$

Multiplying by the in-flight count means that a busy server scores higher, which spreads load; the random 10% exploration lets a slow server that has recovered receive traffic again.

Failover and health. If a server returns an error or does not answer within 2 s, the proxy retries the request on the other healthy servers one by one. Three failures in a row remove a server from the pool. A removed server is probed at most every 3 s, when traffic arrives, until it answers; it then returns with no latency estimate, so it is tried first. If no healthy server is left, the request fails.

The listing below is illustrative pseudocode of this rule, not the verbatim source.

```
function select(healthy):
    unmeasured = [s in healthy if s.ewma is empty]
    if unmeasured is not empty:
        return least busy server in unmeasured
    if random() < 0.10:                          # exploration
        return uniformly random server in healthy
    return server in healthy with minimum  s.ewma * (s.inFlight + 1)

function handle(request):
    tried = {}
    loop:
        candidates = healthy servers not in tried
        if candidates is empty: return failure
        s = select(candidates)
        start = now()
        response = forward(s, request, timeout = 2 s)
        if response is ok:
            rtt = now() - start
            s.ewma = rtt if s.ewma is empty else 0.3*rtt + 0.7*s.ewma
            s.consecutiveFailures = 0
            return response
        s.consecutiveFailures += 1
        if s.consecutiveFailures >= 3: mark s down  (probe at most every 3 s)
        tried.add(s)
```

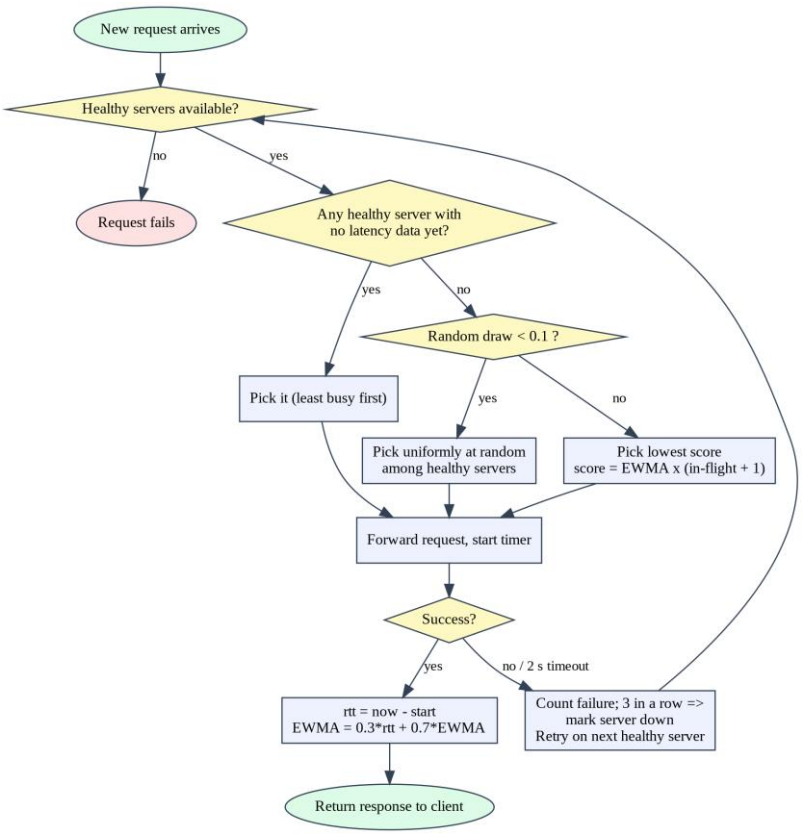


Figure 3.2: Flowchart of server selection, timing and failover

3.4 Novelty statement (what exactly differs from the baseline)

Compared with round robin, the proposed proxy differs in four ways:

1. it **measures** the proxy-to-back-end round-trip time of every request and keeps a smoothed estimate per server, where round robin uses no measurement;
2. it **chooses** by a score that combines that estimate with the current in-flight count, so traffic follows both speed and load;
3. it **explores**: about 10% of requests are random, and a server with no estimate is tried first, so a recovered server is not starved;
4. it **combines** this with retries, failure counting and health probes, and exposes everything on a live dashboard where server delays can be changed and servers taken down during a run.

The EWMA-with-load idea itself is not new (Section 2.1). This implementation uses a plain EWMA over all healthy servers rather than the peak EWMA with two random choices used by Finagle and Envoy.

3.5 Assumptions and design decisions

Table 3.1: Design parameters and reasons

Parameter	Value	Reason
Smoothing factor α	0.3	Damps single spikes but shows a real slowdown within a few requests
Exploration rate	about 10%	Lets a slow server that recovers be sampled again; costs some latency
Score	EWMA × (in-flight + 1)	A busy server scores higher, so load spreads (the score is ms × requests, not plain ms)
Request timeout	2 s	Bounds the wait on an unresponsive server before retrying elsewhere
Failure threshold	3 in a row	Avoids removing a server because of one failed request
Probe interval	at most every 3 s	Probes run when traffic arrives; no background timer

Other decisions and assumptions: the back-ends are simulated so that delays are controllable and runs are repeatable; routing state is kept in memory for simplicity, which limits the design to one proxy instance; and latency means the proxy-to-back-end interval, which in this demo includes the simulated delay (base ± 10% jitter + 5 ms per request in flight) and the local HTTP hop.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Table 4.1: Technology stack

Layer	Technology	Purpose
Framework	Next.js 14.2.15	Dashboard pages and proxy API routes
User interface	React 18.3.1	Live traffic view, charts, controls
Language	TypeScript 5.5.4	Typed proxy, strategy and server code
Styling	Tailwind CSS 3.4.13	Layout and component styling
Animation	framer-motion 11.11.1	Packet animation in the topology diagram

The project runs in development mode: `npm install`, then `npm run dev`, then open `http://localhost:3000`. Node.js version and operating system: [to be filled]. Test machine (CPU, RAM): [to be filled]. All experiments ran on a single machine, with the browser, proxy and simulated servers on the same host.

4.2 Simulation or testbed setup (topology, configuration, parameter table)

The testbed is the architecture of Figure 3.1. Traffic is generated by the dashboard's traffic simulator at a configured rate in packets per second; each packet is sent through the proxy API route to one of the three simulated back-ends. The simulated delay of a back-end is its configured base delay with ±10% jitter plus 5 ms for each request already in flight. Server delays can be changed with a slider and a server can be taken down and brought back from the dashboard, which is how the slowdown and failover experiments were run.

Table 4.2: Experiment configuration

Parameter	Value
Servers and base delays (E1)	Server A 80 ms, Server B 150 ms, Server C 250 ms
Traffic rate and duration (E1)	10 packets/s configured, 30 s per run
Repeats	E1: 3 runs per strategy; E2: 2 runs; E3: 2 runs
Strategies compared	Round robin (baseline) and adaptive (lowest-latency)
EWMA α, exploration, timeout	0.3, about 10%, 2 s
Failure threshold, probe interval	3 consecutive failures, at most every 3 s
Recent-request log	Last 30 requests kept on the dashboard

4.3 Code structure and explanation of key modules

Figure 4.1 shows the source tree of the project generated from the project folder; the number in brackets is the number of lines of code in each file.

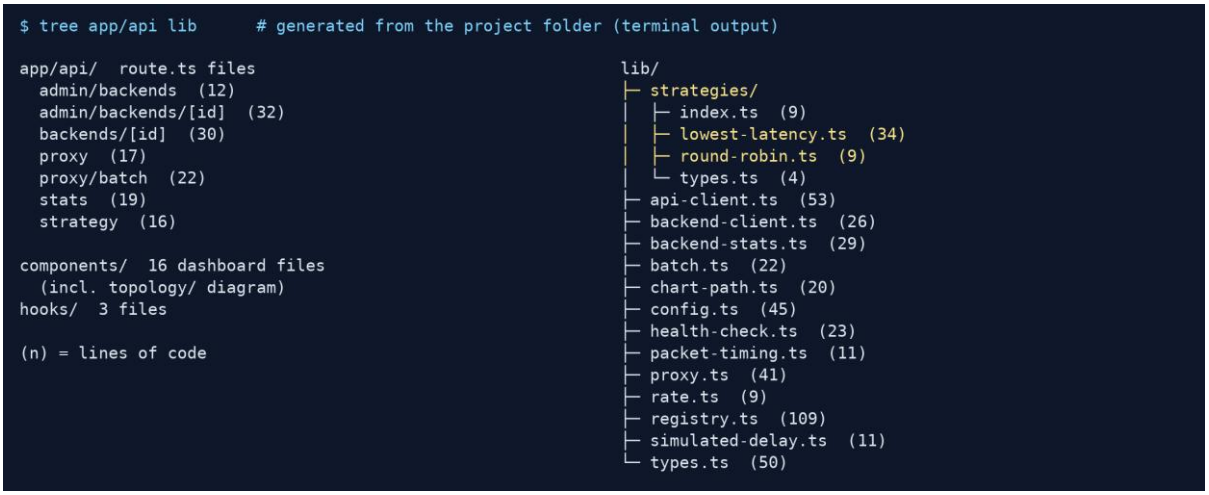


Figure 4.1: Source layout of app/api and lib (terminal output)

- `app/api/` holds the route handlers: proxy, batch, backend, admin, stats and strategy.
- `components/` and `hooks/` hold the dashboard UI (16 files, including the topology diagram) and three hooks for polling and the traffic simulator.
- `lib/` holds the proxy and its supporting modules: `proxy.ts`, `backend-client.ts`, `backend-stats.ts`, `health-check.ts`, `registry.ts` and `config.ts`, among others.
- `lib/strategies/` holds one file per routing strategy. The adaptive strategy, `lowest-latency.ts` (34 lines), and the baseline, `round-robin.ts` (9 lines), are separate small files, with their shared types in `types.ts`.

The very small size of the two strategy files shows how little code the adaptive rule needs once the proxy already measures round-trip time and tracks in-flight requests.

4.4 Challenges faced and how they were solved

Table 4.3: Design issues and how they were handled

Issue	How it was handled
A slow server could be starved once its estimate is high, and a recovered server has no estimate	About 10% random exploration, and a server with no latency data is tried first
A server failing or timing out must not break the client's request	Retry on the other healthy servers; remove a server after 3 consecutive failures; probe it later
Single slow replies could distort routing	EWMA with $\alpha = 0.3$ damps spikes while still showing a real slowdown within a few requests
Latency can be misread as total client latency	Defined and reported throughout as the proxy-to-back-end interval
The first run after a server start is slower (development-mode warm-up)	Results are reported per run, not hidden in an average, and the effect is stated
Probes only run when traffic arrives	Accepted as a limitation and listed under future work (timer-based probes)

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

- **Requests sent / served / failed** – shows whether the proxy kept serving under each strategy.
- **Share of requests per server** – shows directly whether traffic moved towards the faster servers.
- **Mean proxy-measured latency** – the average of the proxy-to-back-end round-trip time over a run; the main measure of benefit.
- **p95 proxy-measured latency** – the 95th percentile over every request in the run; shows the tail, which the mean hides.
- **EWMA per server** – shows how quickly the estimate follows a change in server speed.
- **Failover timing and client-visible failures** – time to mark a server down and to bring it back, and whether any client request failed.

5.2 Scenarios and parameter values

Table 5.1: Experiment scenarios

ID	Scenario	Procedure
E1	Round robin versus adaptive	10 packets/s configured for 30 s; delays A 80 / B 150 / C 250 ms; 3 runs per strategy (run labels such as E1-RR-2 and E1-AD-2)
E2	Slowing one server (adaptive)	Reset; 25 s at 10 packets/s; set Server A's slider to 400 ms; 25 s more with no reset in between; runs r1 and r2 in separate sessions
E3	Failover (adaptive)	10 packets/s; Take down a server, then Bring up; 2 runs (capture from run E3-r2)

Load was held at 10 packets/s in every scenario, so the low, medium and high load rows of the usual results template are not reported. Varying the load is listed as future work.

5.3 Results: tables and graphs, baseline vs proposed (mean of repeated runs)

E1: round robin versus adaptive

Table 5.2 gives the raw values of the three runs of each strategy. Run 1 of each strategy came first after server start and is slower because of development-mode warm-up.

Table 5.2: E1 raw results per run (runs 1 / 2 / 3)

Metric	Round robin	Adaptive
Requests sent / served / failed	302/302/0 · 302/302/0 · 300/300/0	303/303/0 · 300/300/0 · 300/300/0
Share to Server A (fastest)	33% · 33% · 33%	59% · 68% · 67%
Share to Server C (slowest)	33% · 33% · 33%	8% · 5% · 4%
Mean proxy-measured latency	246 · 211 · 212 ms	191 · 149 · 150 ms
p95 proxy-measured latency	393 · 327 · 323 ms	347 · 255 · 239 ms

Table 5.3 summarises the runs as mean ± standard deviation (sample standard deviation over n = 3 runs). The improvement is computed on the means.

Table 5.3: E1 summary, baseline versus proposed (mean ± SD, n = 3)

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improvement (%)
E1 (10 pkt/s)	Mean latency (ms)	223.0 ± 19.9	163.3 ± 24.0	26.8
E1 (10 pkt/s)	p95 latency (ms)	347.7 ± 39.3	280.3 ± 58.3	19.4
E1 (10 pkt/s)	Share to fastest server (%)	33 ± 0	64.7 ± 4.9	+31.7 points
E1 (10 pkt/s)	Share to slowest server (%)	33 ± 0	5.7 ± 2.1	−27.3 points

The per-pair reductions in mean latency were 22%, 29% and 29% (runs 1, 2, 3). Figure 5.1 plots the latency results and Figure 5.2 shows how the 300 requests of one pair of runs were distributed.

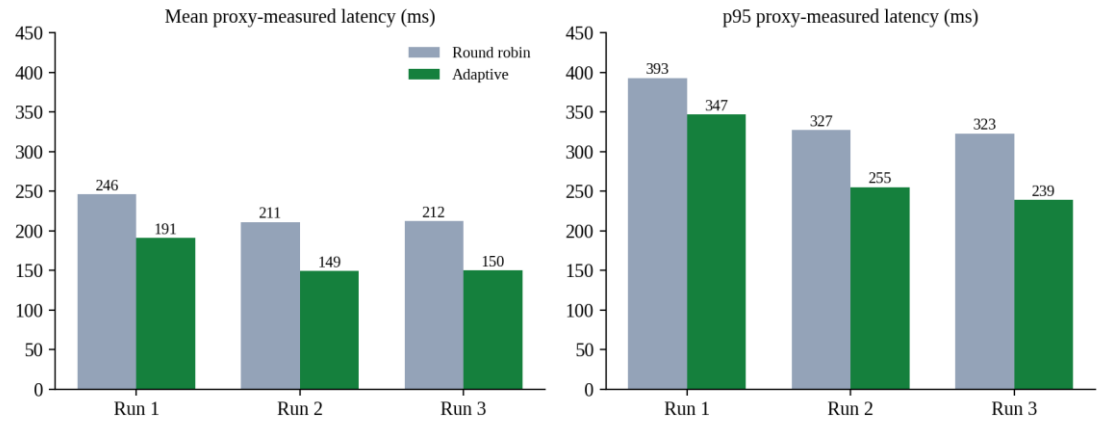


Figure 5.1: Mean and p95 proxy-measured latency per run, round robin versus adaptive (E1)

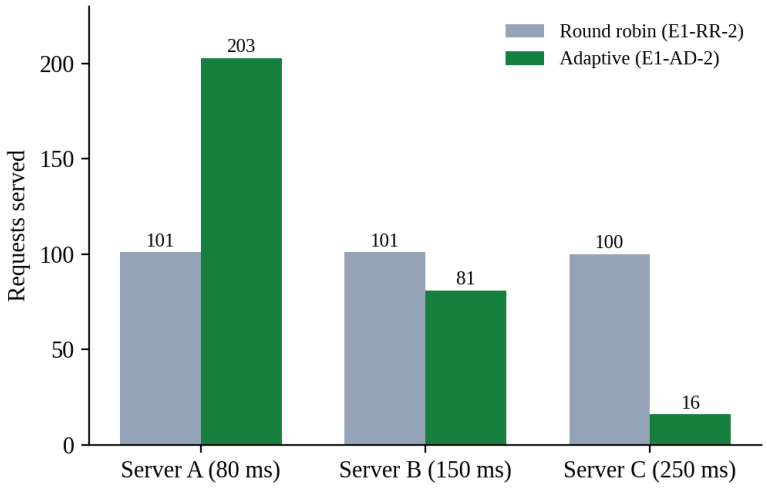


Figure 5.2: Requests served per server in runs E1-RR-2 (302 sent) and E1-AD-2 (300 sent)

Figure 5.3 is the dashboard's recent-requests table during adaptive run E1-AD-2, and Figure 5.4 shows the moving average per server in the same run.

Recent requests

Newest first. The last 30 requests are kept.

Time	Routed to	Response time	Strategy	Failover
9:10:25 PM	Server C	397 ms	lowest-latency	No
9:10:25 PM	Server B	264 ms	lowest-latency	No
9:10:25 PM	Server B	254 ms	lowest-latency	No
9:10:25 PM	Server B	260 ms	lowest-latency	No
9:10:25 PM	Server B	269 ms	lowest-latency	No
9:10:25 PM	Server B	250 ms	lowest-latency	No
9:10:25 PM	Server A	247 ms	lowest-latency	No
9:10:25 PM	Server B	278 ms	lowest-latency	No

Figure 5.3: Recent requests during adaptive run E1-AD-2 (dashboard capture)



Figure 5.4: EWMA per server over time, adaptive run E1-AD-2 (dashboard capture)

E2: slowing one server

Table 5.4: E2 results (adaptive, 10 pkt/s, Server A slowed from 80 ms to 400 ms)

Measure	Run r1 before → after	Run r2 before → after	Note
Server A configured delay	80 → 400 ms	80 → 400 ms	slider
EWMA A / B / C at capture (ms)	123 / 198 / 361 → 450 / 192 / 298	A: 116 → 428	A rises to about 430–450
Cumulative share A / B / C (%)	64 / 30 / 6 → 39 / 45 / 16	66 / 29 / 5 → 39 / 44 / 17	lags: includes pre-change requests
Share of the 250 requests sent after the change (%)	14 / 60 / 27	13 / 58 / 29	isolates new traffic
Mean proxy-measured latency	173 → 258 ms	153 → 258 ms	each phase only

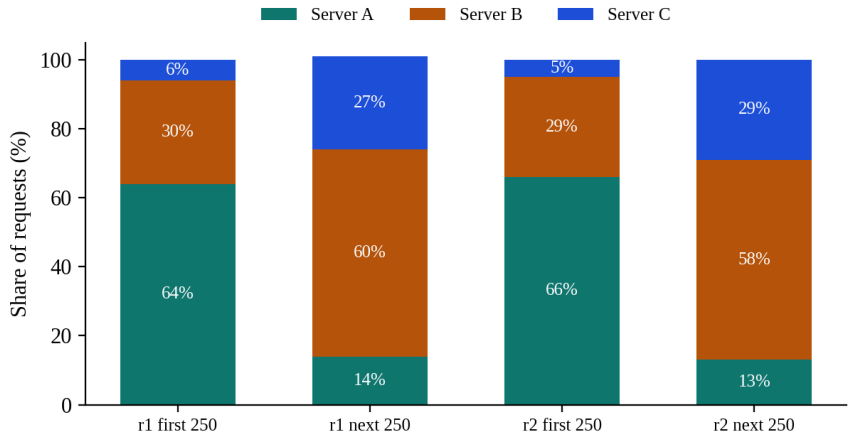


Figure 5.5: Share of requests per server in the first 250 requests and the next 250 after Server A was slowed (E2)

E3: failover

In two runs at 10 packets/s, a server was marked down 0.7–1.2 s after Take down and was back in the pool 0.4–1.3 s after Bring up. There were 0 client-visible failures: 4 failed attempts occurred and all were retried on other servers. Figures 5.6 and 5.7 are captures from run E3-r2.

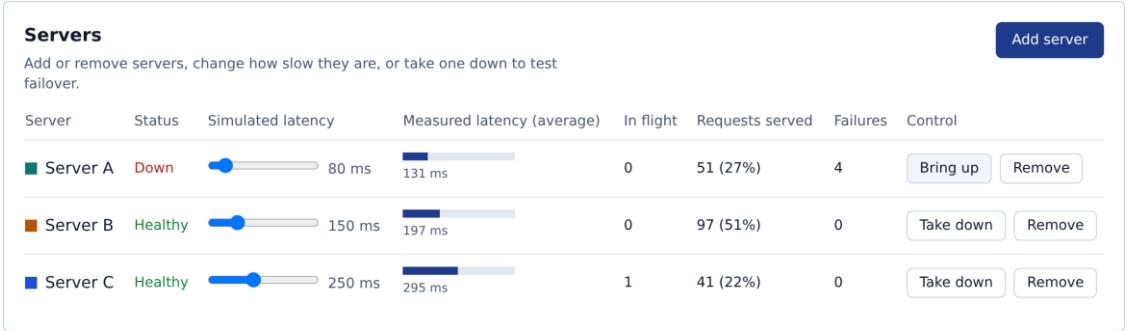


Figure 5.6: After Take down: Server A is Down with 4 failed attempts (run E3-r2)

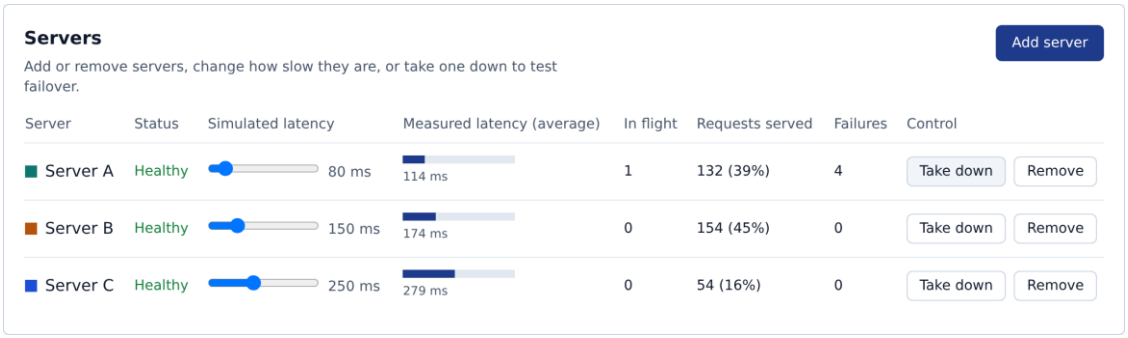


Figure 5.7: After Bring up: Server A is Healthy again (run E3-r2)

5.4 Analysis and discussion (reasons, trends, trade-offs)

Traffic follows speed. Round robin split the requests one third each. The adaptive proxy sent 59–68% to the fastest server and only 4–8% to the slowest, because the slowest server's EWMA, and therefore its score, stayed highest. Moving requests away from the 250 ms server is what lowered the mean: 223 ms to 163 ms on average, a 27% reduction.

Measured latency is higher than the configured delays. The configured delays average 160 ms, but the measured round-robin means are 211–246 ms. The measurement covers the whole proxy-to-back-end call, which includes the local HTTP hop and the extra 5 ms per request in flight, so it is not directly comparable with the slider values.

The tail improves less than the mean. The p95 improved by 19% on average (12%, 22% and 26% per pair) and the spread between runs was larger. This is consistent with the exploration rate and load spreading: adaptive still sent 4–8% of requests to Server C, and a share of that size places the 95th percentile close to Server C's latency.

Reaction to change. After Server A was slowed to 400 ms, its EWMA rose to about 430–450 ms and its share of new requests fell from 64–66% to 13–14%, while Server B, now the lowest, took about 60%. A did not drop to zero because of the 10% random exploration and the load-spreading term; this is the intended trade-off, since it is how a recovered server is noticed. The cumulative shares changed less than the new-traffic shares because they include the earlier requests.

Failover. Failed attempts were absorbed by retries, so clients saw no errors, and the server left and re-entered the pool within about a second or two.

Trade-offs. A larger α reacts faster but follows noise; a smaller α is steadier but slower to notice a change. More exploration finds recoveries sooner but sends more requests to slow servers. Counting load in the score spreads traffic but, because the in-flight count also affects the measured delay, counts load twice.

5.5 Critical evaluation: strengths, weaknesses, conditions where the proposed scheme fails

Strengths. The method is small (one stored number per server and a 34-line strategy), reacts to a slowdown within a few requests, needs no knowledge of server capacity, and was better than round robin on mean latency in all three run pairs.

Weaknesses. The evidence is three runs on one machine with simulated servers, at one load level and with a single delay configuration, so it is a demonstration and not statistical proof. The standard deviations are large relative to the p95 difference. Run 1 of each strategy is affected by warm-up. The latency definition excludes the client–proxy leg.

Conditions where the scheme is expected to do poorly or is untested. (a) With similar server speeds there is little to gain and exploration only adds a small cost. (b) The state is in memory, so several proxy instances would each learn separately and a restart loses everything. (c) A server that fails very quickly can look fast to a latency-based rule; here failures are counted and a server is removed after three in a row, but fast error responses were not tested, and the 2 s timeout path was not tested either. (d) Health probes run only when traffic arrives, so an idle pool is not checked. (e) Behaviour at other loads and with real back-ends was not measured.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings (quantified)

We implemented a reverse proxy that routes by measured latency (EWMA, $\alpha = 0.3$) and load, with 10% exploration, retries, health probes and a live dashboard. In three runs against round robin with simulated servers (80, 150 and 250 ms):

- adaptive routing sent 59–68% of requests to the fastest server, against 33% for round robin;
- mean proxy-measured latency was lower in every pair, 163 ms versus 223 ms on average (27% lower; for example 150 ms versus 212 ms in run 3), and p95 was 19% lower on average;
- after Server A was slowed to 400 ms, it received 13–14% of the next 250 requests, down from 64–66%;
- taking a server down caused no client-visible failures in two runs, and it was out and back within about a second or two.

This is a limited demonstration, not a production benchmark or proof for all workloads.

6.2 Limitations

- State is in memory (lost on restart) and there is one proxy instance only.
- Back-ends are simulated inside the same app; there are no real upstream URLs.
- The in-flight count is kept by the simulated back-end, and load is counted in both the latency and the score.
- Probes run only when traffic arrives; the 2 s timeout path was not tested.
- There is no built-in p95 or throughput metric; the results are three runs on one machine at one load level.

6.3 Future enhancements

Proposed, not implemented:

- shared state (for example Redis) so several proxy instances can coordinate; persistence would depend on its configuration;
- forwarding to real upstream URLs and counting in-flight requests in the proxy itself;
- a built-in p95 and throughput benchmark with more runs and several load levels;
- weighted routing, TLS termination and timer-based health probes.

7. REFERENCES

[1] V. Jacobson, “Congestion avoidance and control,” ACM SIGCOMM Computer Communication Review, vol. 18, no. 4, pp. 314–329, Aug. 1988.

[2] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing TCP’s Retransmission Timer,” RFC 6298, Internet Engineering Task Force, Jun. 2011.

[3] M. Mitzenmacher, “The power of two choices in randomized load balancing,” IEEE Transactions on Parallel and Distributed Systems, vol. 12, no. 10, pp. 1094–1104, Oct. 2001.

[4] V. Cardellini, M. Colajanni, and P. S. Yu, “Dynamic load balancing on web-server systems,” IEEE Internet Computing, vol. 3, no. 3, pp. 28–39, May–Jun. 1999.

[5] Linkerd, “Beyond Round Robin: Load Balancing for Latency,” Linkerd blog, Mar. 16, 2016. [Online]. Available: https://linkerd.io/2016/03/16/beyond-round-robin-load-balancing-for-latency/

[6] Envoy Project, “Peak EWMA Load Balancer,” Envoy proxy documentation. [Online]. Available: https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/contrib/load_balancing_policies/peak_ewma/peak_ewma

[7] NGINX, “Module ngx_http_upstream_module,” NGINX documentation. [Online]. Available: https://nginx.org/en/docs/http/ngx_http_upstream_module.html

Appendix A: Important source code or configuration listings

Configuration values used in all experiments:

```
alpha (EWMA smoothing)         = 0.3
exploration probability        = about 0.10 (uniform over healthy servers)
score                          = ewma_ms * (in_flight + 1)
request timeout                = 2 s
failures before removal        = 3 in a row
probe interval (when traffic) = at most every 3 s
simulated delay                = base +/- 10% jitter + 5 ms per request in flight
E1 base delays                 = A 80 ms, B 150 ms, C 250 ms
E1 traffic                     = 10 packets/s configured, 30 s
```

The selection and forwarding logic is given as pseudocode in Section 3.3. Final source listings of `lib/strategies/lowest-latency.ts`, `lib/strategies/round-robin.ts` and `lib/proxy.ts`: [paste from the project repository before submission].

Appendix B: Additional screenshots, packet captures, raw result tables

The raw per-run results of E1 are in Table 5.2 and those of E2 in Table 5.4. The values behind the summary statistics in Table 5.3 are listed below.

Table B.1: Values used for the E1 summary statistics

Metric	Run 1	Run 2	Run 3	Mean ± SD
Round robin, mean latency (ms)	246	211	212	223.0 ± 19.9
Adaptive, mean latency (ms)	191	149	150	163.3 ± 24.0
Round robin, p95 latency (ms)	393	327	323	347.7 ± 39.3
Adaptive, p95 latency (ms)	347	255	239	280.3 ± 58.3
Adaptive, share to Server A (%)	59	68	67	64.7 ± 4.9
Adaptive, share to Server C (%)	8	5	4	5.7 ± 2.1

Run counts for the E1 pair used in Figure 5.2: round robin E1-RR-2 sent 302, served 302, failed 0 (A 101, B 101, C 100); adaptive E1-AD-2 sent 300, served 300, failed 0 (A 203, B 81, C 16). Dashboard captures are in Figures 5.3, 5.4, 5.6 and 5.7.